2026-6-25 Linux开发工具--权限

- 1. 从 Shell 理解 Linux 权限
  - 1.1 命令首先是一个文件
    - 1.1.1 命令、可执行文件与路径
    - 1.1.2 Shell 如何找到一条命令
    - 1.1.3 为什么常见的命令不能随便写进系统目录
  - 1.2 Shell 在操作系统中的位置
    - 1.2.1 广义操作系统与狭义操作系统
    - 1.2.2 为什么需要外壳
    - 1.2.3 Shell 为什么常用子进程执行命令
  - 1.3 Shell 与权限的连接点
    - 1.3.1 执行命令也要检查文件权限
    - 1.3.2 命令运行时使用的仍然是当前用户身份
    - 1.3.3 从系统文件看拥有者
    - 1.3.4 软件安装为什么需要 sudo
- 2. Linux 权限的基本概念
  - 2.1 什么是权限
    - 2.1.1 权限回答“能还是不能”
    - 2.1.2 为什么 Linux 需要权限
  - 2.2 权限 = 人 + 文件属性
    - 2.2.1 两个条件缺一不可
    - 2.2.2 Linux 下一切皆文件
  - 2.3 权限判断的基本过程
    - 2.3.1 身份、操作与资源
    - 2.3.2 为什么不能只看权限字符
- 3. 用户身份与权限主体
  - 3.1 root 与普通用户
    - 3.1.1 超级用户 root
    - 3.1.2 普通用户
    - 3.1.3 系统用户
  - 3.2 使用 su 切换身份
    - 3.2.1 切换到指定用户
    - 3.2.2 谁需要输入谁的密码
    - 3.2.3 退出切换后的身份
    - 3.2.4 日常使用建议
  - 3.3 使用 sudo 临时提权

        - 3.3.1 sudo 的作用
        - 3.3.2 为什么输入的是自己的密码
        - 3.3.3 sudoers 名单式管理
        - 3.3.4 sudo 不等于永久变成 root
- 4. 文件的角色与权限属性
    - 4.1 Linux 的三类文件角色
        - 4.1.1 拥有者、所属组、其他人
        - 4.1.2 为什么要有用户组
        - 4.1.3 拥有者和所属组可以相同
    - 4.2 用 ls -l 读懂文件权限
        - 4.2.1 权限字符串的构成
        - 4.2.2 文件类型字符
        - 4.2.3 后续字段
    - 4.3 r、w、x 对普通文件的含义
        - 4.3.1 基本含义
        - 4.3.2 可执行权限不等于一定能执行
        - 4.3.3 Linux 更看重内容格式，而不是后缀
    - 4.4 权限只匹配一次
        - 4.4.1 拥有者优先
        - 4.4.2 一个容易误判的实验
        - 4.4.3 root 的特殊行为
- 5. 修改文件权限与归属
    - 5.1 查看文件属性
        - 5.1.1 基本查看方式
        - 5.1.2 查看自己的身份
    - 5.2 chmod：修改权限位
        - 5.2.1 符号方式
        - 5.2.2 数字方式
        - 5.2.3 递归修改
        - 5.2.4 权限检查只会匹配一次
    - 5.3 chown：修改拥有者
        - 5.3.1 基本用法
        - 5.3.2 为什么通常需要 root
        - 5.3.3 修改拥有者和所属组会改变角色映射
    - 5.4 chgrp：修改所属组
        - 5.4.1 基本用法
        - 5.4.2 普通用户能否修改所属组

- 5.5 修改权限后的验证
- 6. 目录权限与删除逻辑
  - 6.1 目录也是文件
    - 6.1.1 目录保存文件名与文件对象的对应关系
    - 6.1.2 目录的 r、w、x
  - 6.2 进入目录需要什么权限
    - 6.2.1 x 是进入目录的关键
    - 6.2.2 只有 r 没有 x
    - 6.2.3 路径上的每一级目录都要能进入
  - 6.3 列出目录内容需要什么权限
    - 6.3.1 r 负责读取文件名
    - 6.3.2 查看详细信息还需要 x
    - 6.3.3 w 负责修改目录项
  - 6.4 删除文件看父目录
    - 6.4.1 删除的是目录项
    - 6.4.2 一个反直觉的课堂实验
    - 6.4.3 root 文件的删除
  - 6.5 目录权限的常见组合
- 7. 新建文件的默认权限与 umask
  - 7.1 为什么新建文件不是 rwxrwxrwx
    - 7.1.1 普通文件的起始权限
    - 7.1.2 目录的起始权限
    - 7.1.3 起始权限还要经过 umask
  - 7.2 umask 是什么
    - 7.2.1 权限掩码
    - 7.2.2 最终权限公式
    - 7.2.3 为什么不能用普通减法理解
  - 7.3 umask 计算示例
    - 7.3.1 普通文件
    - 7.3.2 目录
    - 7.3.3 umask 为 000
  - 7.4 umask 的作用范围
    - 7.4.1 当前进程属性
    - 7.4.2 想长期生效
    - 7.4.3 常见 umask
- 8. 共享目录与粘滞位
  - 8.1 多人共享目录的问题

        - 8.1.1 为什么不放在某人的家目录
        - 8.1.2 只开放目录写权限会产生什么结果
    - 8.2 粘滞位解决什么问题
        - 8.2.1 目标
        - 8.2.2 查看粘滞位
        - 8.2.3 谁的删除操作会被允许
        - 8.2.4 粘滞位不是“把文件变成只读”
    - 8.3 /tmp 为什么特殊
- 9. 权限问题排查与课堂练习
    - 9.1 权限被拒绝时怎么查
        - 9.1.1 先确认身份
        - 9.1.2 再确认文件和目录
        - 9.1.3 检查完整路径
        - 9.1.4 判断操作对象
    - 9.2 三个典型场景
        - 9.2.1 能列出目录，却不能进入
        - 9.2.2 文件可读，却无法执行
        - 9.2.3 文件本身不可写，却能被删除
    - 9.3 权限判断的完整顺序
    - 9.4 常用命令速查
        - 9.4.1 查看身份与权限
        - 9.4.2 修改权限与归属
        - 9.4.3 默认权限与共享目录
        - 9.4.4 身份切换
    - 9.5 课堂自测问题
        - 9.5.1 用户身份
        - 9.5.2 文件属性
        - 9.5.3 目录与删除
        - 9.5.4 umask
    - 9.6 本篇结论

1. 从 Shell 理解 Linux 权限

1.1 命令首先是一个文件

1.1.1 命令、可执行文件与路径

Linux 中学到的很多“命令”，本质上都是系统中的可执行文件。例如：

```
which ls
ls -l /usr/bin/ls
```

可以看到 `ls` 位于 `/usr/bin/ls`，类似：

```
-rwxr-xr-x 1 root root 117608 ... /usr/bin/ls
```

这段输出同时包含了几类重要信息：

- 第一个 `-`：它是普通文件。
- `rwx`：拥有者 `root` 可以读、写、执行。
- `r-x`：所属组可以读、执行，但不能修改。
- `r-x`：其他用户可以读、执行，但不能修改。
- `root root`：文件的拥有者与所属组都是 `root`。
  因此，普通用户平时虽然可以执行 `ls`，却不能随意修改 `/usr/bin/ls`。这说明“命令能用”和“命令文件能改”是两种不同的权限问题。
  Linux 系统通常把常用命令放在 `/usr/bin`、`/bin`、`/usr/sbin`、`/sbin`、`/usr/local/bin` 等目录中。可以查找一个命令来自哪里：

```
which ls
whereis ls
```

也可以在目录树中按文件名查找：

```
find /usr/bin -name 'ls'
find / -name 'some_file' 2>/dev/null
```

`which` 主要按照当前 Shell 的 `PATH` 查找命令；`find` 则是遍历文件系统目录树。二者解决的问题不同。

1.1.2 Shell 如何找到一条命令

输入一条命令后，Shell 并不是凭空“知道”程序在哪里。它大致会按以下顺序处理：

1. 读取用户输入的命令行。
2. 判断是否存在别名、函数或 Shell 内建命令。
3. 如果不是内建命令，就根据 `PATH` 指定的目录查找外部可执行文件。
4. 找到目标文件后，通常创建子进程，由子进程加载并执行该程序。
   查看 `PATH`：

```
echo "$PATH"
```

`PATH` 是由多个目录组成的搜索列表。Shell 依次到这些目录中寻找命令文件。
别名是 Shell 内存中的一组替换规则，例如：

```
alias ll='ls -l'
ll
```

别名只影响当前 Shell 会话，重新登录或打开新的 Shell 后通常不会继续存在。别名也不是磁盘上的文件，而 `ls` 本身是真实的系统程序，两者不能混为一谈。
如果 Shell 在别名、内建命令和 `PATH` 中都找不到目标，就会提示：

```
command not found
```

这不是权限不足，而是“没有找到这个命令”。

1.1.3 为什么常见的命令不能随便写进系统目录

不建议把随便编写的程序放入 `/usr/bin`。原因包括：

- 容易污染系统命令空间。
- 可能与系统自带命令重名。
- 系统升级时可能被覆盖或产生冲突。
- 普通用户不应拥有修改系统命令目录的权限。
  自己的程序更适合放在用户目录、项目目录或专门的 `~/bin` 中，再把该目录加入 `PATH`。

1.2 Shell 在操作系统中的位置

1.2.1 广义操作系统与狭义操作系统

从广义上看，一个操作系统除了内核，还包括：

```
内核 + Shell/图形界面 + 必要的基础应用
```

从狭义上看，核心是操作系统内核：

```
Linux 内核
```

Shell 是操作系统外面的一层软件，也是用户和内核之间的命令行接口：

```
用户
  ↓ 输入命令
Shell
  ↓ 解析并请求系统调用
Linux 内核
  ↓ 管理 CPU、内存、文件、设备
硬件
```

内核执行完操作后，结果再沿相反方向返回给 Shell，Shell 最终把结果显示给用户。

1.2.2 为什么需要外壳

用户一般不直接与内核交互。内核提供的是系统调用等底层接口，不适合直接手工调用；
Shell 把用户容易理解的命令翻译成内核能够处理的请求。
例如用户输入：

```
ls -l
```

Shell 负责解析出：

```
程序：ls
参数：-l
```

随后找到对应程序并交给内核执行。文件读取、目录访问、权限判断等最终都由内核完成。
可以把 Shell 理解成一层缓冲：

- 用户输入错误命令时，通常只是 Shell 报告错误。
- 外部程序崩溃时，通常是这个子进程退出，Shell 仍可继续接受命令。
- Shell 提供统一、稳定的操作界面，使普通用户不必直接处理内核底层细节。
  但需要注意：真正决定“这个用户能否读写某个文件”的仍然是内核，而不是 Shell。

1.2.3 Shell 为什么常用子进程执行命令

Shell 本身需要持续运行并等待用户输入。如果直接让 Shell 进程变成 `ls`，执行完 `ls` 后原来的 Shell 就消失了。
因此，Shell 通常采用如下方式：

```
Shell 进程
  ├─ 创建子进程
  ├─ 子进程加载外部命令
  ├─ 子进程执行并退出
  └─ Shell 等待并继续接受下一条命令
```

对于 `cd`、`export` 这类会改变 Shell 自身状态的操作，通常会作为 Shell 内建命令执行；对于 `ls`、`cat` 等外部程序，则要找到文件并通过子进程执行。

1.3 Shell 与权限的连接点

1.3.1 执行命令也要检查文件权限

执行一个外部命令之前，系统必须能够读取这个可执行文件，并且当前用户必须具有执行权限。
例如：

```
chmod u-x ./program
./program
```

如果拥有者失去了执行权限，就可能出现：

```
Permission denied
```

即使文件具有执行权限，也不代表它一定能运行。它还必须是 Linux 能识别的可执行格式。普通文本文件即使执行了：

```
chmod +x note.txt
./note.txt
```

也不会真正变成一段可执行程序。
所以“文件能执行”至少需要两个条件：

1. 文件格式合法，例如 ELF 可执行文件或带有合法解释器的脚本。
2. 当前用户具有对应的 `x` 权限。

1.3.2 命令运行时使用的仍然是当前用户身份

普通用户执行 `ls` 时，虽然 `/usr/bin/ls` 属于 `root`，但 `ls` 进程采用的是执行者自己的身份，而不是 root 身份。
因此：

- 程序文件可以被所有用户执行。
- 程序访问哪些文件，仍然受到当前用户权限的限制。
- 程序本身属于 root，并不会让它自动获得 root 的全部权限。
  这也解释了一个常见现象：一个命令明明“能运行”，却在读取某个文件时提示 `Permission denied`。前者是命令文件可执行，后者是运行过程中访问目标资源失败。

1.3.3 从系统文件看拥有者

PDF 中查看软件源配置时出现过类似输出：

```
ll /etc/yum.repos.d/
```

```
-rw-r--r-- 1 root root ... CentOS-Base.repo
```

对普通用户来说，这个配置文件的含义是：

- `r--`：可以读取。
- 没有 `w`：不能修改。
- `root root`：拥有者和所属组都是 root。
  因此普通用户可以查看它，却不能直接编辑它。修改系统配置通常需要使用 `sudo`。

1.3.4 软件安装为什么需要 sudo

课堂手记中说过：

```
普通用户安装软件
→ 软件需要写入系统指定目录
→ 普通用户没有这些目录的写权限
→ 安装失败
```

PDF 中的命令也体现了这一点：

```
sudo yum install -y lrzsz
sudo apt install -y lrzsz
```

这里保留这个例子的目的不是展开讲软件包管理，而是说明权限问题的本质：安装软件需要向系统目录写入内容，所以必须临时获得管理员权限。
普通用户不必长期切换成 root，而可以让 `sudo` 只对一条命令临时授权。这就把 Shell、用户身份、文件权限和最小权限原则连接在了一起。

2. Linux 权限的基本概念

2.1 什么是权限

2.1.1 权限回答“能还是不能”

权限回答的是一个直接问题：

> 某个用户能否对某个资源执行某个操作?

能做，说明拥有对应权限；不能做，说明权限不足。
例如：

- 普通用户能不能读取 `/etc/passwd`？
- 文件拥有者能不能修改自己的文件?
- 一个用户能不能进入某个目录?
- 一个用户能不能删除另一个用户创建的文件?
  每一个问题都同时涉及“谁”“对什么资源”“做什么操作”。

2.1.2 为什么 Linux 需要权限

Linux 是多用户操作系统，同一台服务器可能允许多个用户登录。如果没有权限控制，一个普通用户可以删除系统文件、修改其他用户的数据或执行危险操作，系统将无法稳定运行。
权限的主要目的包括：

1. 保护系统关键文件。
2. 区分不同用户的数据范围。
3. 防止误操作和恶意操作。
4. 支持多用户共享同一台机器。
   权限不是为了让操作“变得更麻烦”，而是为了在共享环境中建立稳定的边界。

2.2 权限 = 人 + 文件属性

2.2.1 两个条件缺一不可

课堂中把权限概括为：

```
权限 = 人 + 文件属性
```

这里的“人”不是抽象的某个人，而是当前进程实际使用的用户身份；“文件属性”是文件在磁盘上记录的权限模式。
只有对应身份拥有对应属性，操作才能通过内核检查。
例如一个文件权限为：

```
-rw-r--r--
```

文件拥有者可以读写；所属组和其他人只能读取。

2.2.2 Linux 下一切皆文件

Linux 把许多资源统一抽象成文件：

- 普通文件。
- 目录。
- 字符设备。

- 块设备。
- 管道。
- 符号链接。
- 套接字。
  因此，权限不仅适用于源代码和文本文件，也适用于设备、目录等资源。

2.3 权限判断的基本过程

2.3.1 身份、操作与资源

判断权限时可以依次问三个问题：

```
谁在执行？ → 对象是什么文件？ → 要执行什么操作？
```

例如：

```
用户 whb
对 /home/whb/test.c
执行写入操作
```

内核会检查用户相对于该文件的角色，再检查角色是否具有写权限。

2.3.2 为什么不能只看权限字符

看到权限字符还不够，还要确认：

- 当前用户是谁。
- 当前用户是否属于文件所属组。
- 操作的是文件内容还是父目录中的目录项。
- 路径上的每一级目录是否都可进入。
- 是否涉及粘滞位等额外规则。
- 当前进程是否拥有 root 等高权限身份。
  因此，权限问题经常需要结合 `ls -l`、`id`、`namei` 等命令一起判断。

3. 用户身份与权限主体

3.1 root 与普通用户

3.1.1 超级用户 root

root 是 Linux 中的超级用户，通常拥有最高权限：

- 可以管理用户和用户组。
- 可以修改系统文件。
- 可以改变文件拥有者。
- 在普通 DAC 权限模型中，通常可以绕过一般文件权限限制。
- 可以在不需要目标用户密码的情况下切换为其他用户。
  Linux 中一般只有一个 root 账号。知道 root 密码，通常就意味着拥有系统的管理能力。

3.1.2 普通用户

普通用户受权限规则限制，通常只能：

- 操作自己的家目录。
- 读取系统公开配置。
- 执行系统允许执行的命令。
- 在自己有写权限的目录中创建文件。
  普通用户想安装软件、修改系统配置或操作其他用户的资源，通常需要提权。

3.1.3 系统用户

系统中还存在一些专门给服务使用的账号，例如运行 Web 服务、数据库服务的用户。它们通常没有正常交互式登录环境，也不应作为日常开发账号使用。
系统中的用户和组信息可以从以下文件查看：

```
cat /etc/passwd
cat /etc/group
```

当前用户和组信息：

```
id
groups
whoami
```

3.2 使用 su 切换身份

3.2.1 切换到指定用户

```
su username
```

这会启动一个新的 Shell，并以目标用户身份运行。
使用登录 Shell：

```
su - username
```

区别是：

- `su username`：尽量保持当前环境，只切换用户身份。
- `su - username`：按照目标用户重新登录，加载其家目录和登录环境。
  切换到 root：

```
su -
```

不写用户名时，`su` 通常默认目标用户是 root，因此需要输入 root 密码。

3.2.2 谁需要输入谁的密码

- 普通用户切换到 root：通常需要 root 密码。
- 普通用户切换到另一个普通用户：通常需要目标用户的密码。
- root 切换到任何用户：一般不需要密码。
  这也是为什么“知道 root 密码”基本等于拥有系统管理权。

3.2.3 退出切换后的身份

执行：

```
exit
```

或者在 Shell 中按 `Ctrl+D`，可以退出当前登录 Shell，回到原来的身份。
注意：`Ctrl+D` 的本质是向当前 Shell 发送文件结束信号，使 Shell 退出；它不是直接修改文件权限，也不是一个“权限切换开关”。

3.2.4 日常使用建议

最稳妥的做法是每个用户使用自己的普通账号。只有确实需要系统管理时，才临时提升权限。长期使用 root 会扩大误操作的影响范围，例如一条路径写错的 `rm` 命令就可能删除普通用户无法删除的系统文件。

3.3 使用 sudo 临时提权

3.3.1 sudo 的作用

````
`sudo` 让被授权的普通用户以 root 或其他指定用户身份执行一条命令。
例如：
```bash
sudo apt install -y lrzsz
````

这里只让这一条安装命令获得写入系统目录的能力，不需要长期进入 root Shell。

3.3.2 为什么输入的是自己的密码

普通用户执行 `sudo` 时，默认验证的是当前用户自己的密码，而不是 root 密码。
这样设计的意义是：

- 不把 root 密码交给所有管理员。
- 将权限授予具体普通用户。
- 每次提权可以记录是谁发起的。
  如果输入自己的密码就能绕过所有限制，那 sudo 就失去意义，因此还要配合授权名单。

3.3.3 sudoers 名单式管理

只有被管理员授权的普通用户才能使用 sudo。授权规则通常位于：

```
/etc/sudoers
/etc/sudoers.d/
```
```

可以把这个机制理解为一种“名单”：

- 在名单中的用户或命令，可以按规则执行。
- 不在名单中的操作，仍然被拒绝。
- 可以只授予某些命令，而不是直接给完整 root 权限。
  编辑 sudo 配置时应使用：

```
sudo visudo
```

不要直接随意编辑 `/etc/sudoers`。如果语法错误，可能导致整个系统都无法正常提权。

3.3.4 sudo 不等于永久变成 root

执行：

```
sudo command
```

只对当前命令提权。执行完以后，普通 Shell 仍是普通用户身份。
查看 sudo 授权情况：

```
sudo -l
```

如果需要一次执行多条管理命令，可以使用：

```
sudo bash
sudo -i
```

但应尽量缩小提权范围，遵循最小权限原则。

4. 文件的角色与权限属性

4.1 Linux 的三类文件角色

4.1.1 拥有者、所属组、其他人

每个文件都记录三类角色的权限：

角色	英文	含义
拥有者	owner	文件最初由谁创建，通常归谁所有
所属组	group	文件属于哪个用户组，便于组级授权
其他人	other	既不是拥有者，也不属于所属组的用户

这三种角色不是三个可以同时累加的权限集合。实际判断时，用户通常只会命中其中一类。

4.1.2 为什么要有用户组

用户组可以把多个用户组织在一起，例如项目组、开发组或运维组。

如果多个用户都需要读写同一批项目文件，可以：

1. 创建项目组。
2. 把相关用户加入该组。
3. 把文件所属组设为项目组。
4. 给所属组分配所需权限。

这比逐个给每个用户单独设置权限更容易管理。

4.1.3 拥有者和所属组可以相同

现实中文件拥有者和所属组经常都是同一个用户名。例如：

```
-rw-r--r-- 1 whb whb ... file.txt
```

这表示拥有者是 `whb`，所属组也是 `whb`。但二者仍然是独立属性，只是恰好名称相同。

4.2 用 ls -l 读懂文件权限

4.2.1 权限字符串的构成

执行：

```
ls -l
```

一个典型输出：

```
-rw-r--r-- 1 whb whb 1024 Nov 20 10:00 notes.txt
```

第一部分可以拆成十位：

```
-   rw-   r--   r--
|    |     |     └─ other 权限
|    |     └─────── group 权限
|    └───────────── owner 权限
└────────────────── 文件类型
```

权限顺序固定为 `r`、`w`、`x`。如果某个位置显示 `-`，表示没有对应权限。

4.2.2 文件类型字符

字符	类型	例子
`-`	普通文件	源代码、文本、库、可执行文件
`d`	目录	`/home`、项目目录
`l`	符号链接	指向另一个路径的链接
`p`	命名管道	进程间通信使用的 FIFO
`c`	字符设备	键盘、终端等
`b`	块设备	磁盘等按块访问的设备
`s`	套接字	本地进程通信

文件中常说的“动态库、静态库、可执行文件”在文件类型上通常仍然都是普通文件 `-`，它们的区别来自内容格式与用途，而不是第一个字符。

4.2.3 后续字段

`ls -l` 后面的字段通常包括：

```
权限  硬链接数  拥有者  所属组  文件大小  修改时间  文件名
```

例如：

```
-rw-r--r-- 1 root root 676 Oct 8 20:47 CentOS-Base.repo
```

可以直接看出：

- 普通文件。
- root 可读写，其他用户只读。
- 文件属于 root。
- 文件属于 root 组。

4.3 r、w、x 对普通文件的含义

4.3.1 基本含义

对普通文件：

权限	含义
`r`	读取文件内容
`w`	修改文件内容
`x`	允许内核执行该文件

没有 `r` 时，读取内容通常失败；没有 `w` 时，不能修改内容；没有 `x` 时，不能把文件作为程序执行。

4.3.2 可执行权限不等于一定能执行

给文本文件增加执行权限：

```
chmod +x note.txt
```

并不会把文本变成合法程序。执行时仍可能失败。

一个文件真正能够执行，需要：

1. 文件具有合法的可执行格式或合法解释器声明。
2. 当前用户对该文件具有 `x` 权限。

因此，`chmod +x` 的意思是“当系统尝试执行这个文件时，允许它被执行”，而不是“把它转换成可执行程序”。

4.3.3 Linux 更看重内容格式，而不是后缀

Linux 通常不依靠 `.exe`、`.sh` 这样的后缀来决定文件能不能执行。内核检查的是文件内容和可执行格式。

不过，后缀仍然对人类识别文件类型以及编辑器、编译器、构建工具很有帮助。程序能否执行和工具是否识别后缀是两个不同层面的事情。

4.4 权限只匹配一次

4.4.1 拥有者优先

判断文件权限时，通常按以下顺序：

1. 当前用户是不是拥有者？
2. 如果不是，是否属于文件所属组？
3. 如果仍不是，使用 other 权限。

一旦命中拥有者，就只按拥有者权限判断，不会再因为用户同时也属于文件所属组而叠加 group 权限。

4.4.2 一个容易误判的实验

假设文件的拥有者和所属组都是同一个用户：

```
-rw------- 1 whb whb ... file
```

如果去掉拥有者的读权限：

```
chmod u-r file
```

用户 `whb` 仍会先命中“拥有者”角色，而不会因为自己同时属于 `whb` 组，就转去使用 group 权限。

所以即使拥有者和所属组名称相同，权限也不是两组相加，而是按最先匹配的角色判断。

4.4.3 root 的特殊行为

root 通常可以无视普通文件权限限制。例如 root 可以读取权限为 `000` 的文件，也可以修改多数系统文件。

但 root 也可能受到挂载方式、只读文件系统、安全模块等其他机制约束。不要把“root 基本不受普通文件权限限制”误解为“root 在任何情况下都绝对没有限制”。

5. 修改文件权限与归属

5.1 查看文件属性

5.1.1 基本查看方式

```
ls -l file
ls -ld dir
stat file
```

`ls -l` 适合查看权限、拥有者和所属组；`ls -ld` 查看目录本身的权限；`stat` 还能看到 inode、时间属性和完整模式信息。

5.1.2 查看自己的身份

```
id
groups
```

只有先确认当前用户和组，才能判断自己相对于文件究竟属于 owner、group 还是 other。

5.2 chmod：修改权限位

5.2.1 符号方式

基本形式：

```
chmod [角色][操作][权限] 文件
```

角色：

字符	含义
`u`	user，拥有者
`g`	group，所属组
`o`	other，其他人
`a`	all，三种角色

操作：

字符	含义
`+`	增加权限
`-`	删除权限
`=`	设置为指定权限

例子：

```
chmod u+x script.sh        # 拥有者增加执行权限
chmod g-w file.txt         # 所属组去掉写权限
chmod o+r file.txt         # 其他人增加读权限
chmod a+r file.txt         # 所有角色增加读权限
chmod u=rwx,g=rx,o= file
```

5.2.2 数字方式

`r`、`w`、`x` 分别对应：

```
r = 4
w = 2
x = 1
```

每种角色将三个数字相加：

数字	权限	含义
7	`rwx`	可读、可写、可执行
6	`rw-`	可读、可写
5	`r-x`	可读、可执行
4	`r--`	只读
3	`-wx`	可写、可执行
2	`-w-`	只写
1	`--x`	只执行
0	`---`	无权限

常见设置：

```
chmod 644 file.txt
chmod 755 script.sh
chmod 600 secret.txt
chmod 700 private_dir
```

三组数字依次代表：

```
chmod 拥有者 所属组 其他人
```

例如：

```
644 = rw- r-- r--
755 = rwx r-x r-x
```

5.2.3 递归修改

```
chmod -R 755 project
```

`-R` 会递归影响目录内的所有内容。递归修改前要确认目标路径，并注意该设置是否真的适合所有文件。
对目录和普通文件使用同一套权限不总是合适。例如目录需要 `x` 才能进入，但普通文本通常不需要执行权限。

5.2.4 权限检查只会匹配一次

修改权限时也要记住前面的规则：

- 拥有者、所属组、其他人是三个类。
- 当前用户只使用命中的那一类权限。
- 不属于任何匹配类时，才使用 other。

5.3 chown：修改拥有者

5.3.1 基本用法

```
sudo chown user file
sudo chown user:group file
sudo chown -R user:group dir
```

例子：

```
sudo chown whb:developers project.c
```

修改后，`whb` 成为拥有者，`developers` 成为所属组

5.3.2 为什么通常需要 root

文件拥有者的改变会影响谁能够控制该文件。普通用户不能随便把自己文件的所有权转给另一个用户，因为这涉及把资源交给别人。

通常只有 root 才能把文件的所有权转给任意用户。具体系统配置可能有差异，但“任意修改 owner”属于高权限操作这一原则不变。

5.3.3 修改拥有者和所属组会改变角色映射

如果文件原本属于：

```
owner: alice
group: alice
```

修改为：

```
owner: bob
group: developers
```

那么 alice 可能从拥有者变成 other，bob 变成拥有者，组内成员也会按新组重新判断权限。所以 `chown` 不只是改了一行文字，而是改变了后续权限判断的角色归属。

5.4 chgrp：修改所属组

5.4.1 基本用法

```
chgrp group file
```

例子：

```
chgrp developers project.c
```

命令名是 `chgrp`，不是 `chgroup`。

5.4.2 普通用户能否修改所属组

普通用户通常只能把自己文件的所属组改成自己所属的组。系统管理员可以修改成任意合法组，具体行为还取决于系统配置。

修改所属组后，组成员是否拥有读写权限，仍取决于 group 权限位。

5.5 修改权限后的验证

修改权限后立即查看：

```
ls -l file
stat file
```

不要只根据命令“没有报错”判断是否达到目标。还要确认：

1. 修改的确实是目标文件或目录。
2. owner、group、other 三组权限都符合预期。
3. 当前用户实际属于哪一类角色。
4. 路径上的父目录是否同样允许访问。

6. 目录权限与删除逻辑

6.1 目录也是文件

6.1.1 目录保存文件名与文件对象的对应关系

目录本身是一个特殊用途的文件。它维护的是目录项信息，例如：

```
文件名 → inode
```

因此，目录权限影响的是“能不能查看目录项、能否进入目录、能否增加或删除目录项”，而不是直接等同于普通文件内容的读写。

6.1.2 目录的 r、w、x

权限	对目录的含义
`r`	可以读取目录中的文件名单
`w`	可以创建、删除、重命名目录项
`x`	可以进入目录或穿过目录访问已知路径

这三种权限是分工的，不能把“对目录的 w”简单理解为“修改目录中所有文件的内容”。

6.2 进入目录需要什么权限

6.2.1 x 是进入目录的关键

进入一个目录需要：

```
x
```

没有 `x` 时，即使知道目录中的文件路径，也无法正常穿过这个目录访问文件。

例如：

```
cd private
```

如果没有 `x` 权限，会得到：

```
Permission denied
```

6.2.2 只有 r 没有 x

只有 `r` 而没有 `x` 时：

- 可以读取目录中的部分文件名信息。
- 无法真正进入目录。
- 无法正常访问目录项对应的文件属性或内容。

这也是说“进入目录靠 x，不靠 r”的原因。

6.2.3 路径上的每一级目录都要能进入

访问：

```
/home/alice/project/file.txt
```

系统需要逐级穿过：

```
/
home/
alice/
project/
```

只要其中某一级目录没有 `x` 权限，就可能无法访问最终文件。

可以使用：

```
namei -l /home/alice/project/file.txt
```

查看路径上每一级目录的权限。

6.3 列出目录内容需要什么权限

6.3.1 r 负责读取文件名

目录有 `r` 权限，才可以读取目录项名称。

```
ls dir
```

通常需要目标目录的 `r` 权限。

6.3.2 查看详细信息还需要 x

执行：

```
ls -l dir
```

除了读取文件名，还要获取每个文件的详细属性。如果目录没有 `x`，即使能读到部分名称，也可能无法访问这些条目的元数据。

6.3.3 w 负责修改目录项

目录的 `w` 权限允许修改目录内容，例如：

- 新建文件。
- 删除文件。
- 重命名文件。
- 创建子目录。

但普通创建、删除和重命名目录项通常同时需要父目录的 `w` 和 `x`。

6.4 删除文件看父目录

6.4.1 删除的是目录项

删除文件时，系统需要修改父目录中的目录项。因此，判断能否删除文件，关键在父目录：

```
父目录具有 w + x
→ 可以修改该目录中的目录项
→ 可能删除目录中的文件
```

这并不要求待删除文件自身可写。

6.4.2 一个反直觉的课堂实验

假设 root 在普通用户的目录中创建了一个权限为 `000` 的文件。普通用户可能仍然能够删除它，因为：

- 文件本身没有读写执行权限。
- 但父目录属于普通用户，且具有写和执行权限。
- 删除修改的是父目录的目录项，而不是文件内容。

这个现象从“文件内容权限”角度看很反直觉，但从目录结构和删除机制看是合理的。

6.4.3 root 文件的删除

如果没有粘滞位，普通用户可以删除自己拥有写和执行权限的目录中的 root 创建文件。这再次说明删除权限主要取决于父目录。

如果目录启用了粘滞位，删除规则会进一步受限。

6.5 目录权限的常见组合

权限	能否列目录	能否进入	能否新增/删除条目
`---`	否	否	否
`r--`	可读取部分名称	否	否
`--x`	不能正常列出	已知路径时可能可进入	否
`r-x`	可以	可以	否
`rw-`	可以读取	通常不能正常穿过	不能可靠完成目录项操作
`rwx`	可以	可以	可以

实际行为还会受到文件系统、ACL、挂载选项和父目录权限影响，但上表适合理解基础模型。

7. 新建文件的默认权限与 umask

7.1 为什么新建文件不是 rwxrwxrwx

7.1.1 普通文件的起始权限

普通文件通常从：

```
666 = rw-rw-rw-
```

开始，因为一般新建的文本或源代码不应该默认具有执行权限。

7.1.2 目录的起始权限

目录通常从：

```
777 = rwxrwxrwx
```

开始，因为目录需要执行权限才能进入。

7.1.3 起始权限还要经过 umask

新建文件的实际权限还要经过 umask 过滤，所以最终通常不会看到所有人可写的普通文件或无限制目录。

7.2 umask 是什么

7.2.1 权限掩码

umask 是“权限掩码”，它表示创建文件时要屏蔽掉哪些权限位。

查看当前 umask：

```
umask
```

常见输出：

```
0022
0002
```

7.2.2 最终权限公式

```
最终权限 = 起始权限 & (~umask)
```

也可理解为：

```
最终权限 = 起始权限“屏蔽掉”umask 中出现的权限
```

这里的“去掉”是位运算层面的屏蔽，不是简单减法。

7.2.3 为什么不能用普通减法理解

位运算表达的是：

```
umask 中为 1 的位 → 对应权限被清除
umask 中为 0 的位 → 保留起始权限
```

虽然很多常见例子的结果恰好看起来像减法，但根本机制是按位屏蔽。

7.3 umask 计算示例

7.3.1 普通文件

以起始权限 `666`、umask `022` 为例：

```
666 = rw-rw-rw-
022 = --- -w- -w-

666 & (~022)
= 644
= rw-r--r--
```

创建出的普通文件通常是 `644`。

7.3.2 目录

以起始权限 `777`、umask `022` 为例：

```
777 = rwxrwxrwx
022 = --- -w- -w-

777 & (~022)
= 755
= rwxr-xr-x
```

创建出的目录通常是 `755`。

7.3.3 umask 为 000

临时执行：

```
umask 000
touch test.txt
mkdir testdir
ls -ld test.txt testdir
```

如果起始权限没有被其他规则修改，通常可以看到：

```
-rw-rw-rw- test.txt
drwxrwxrwx testdir
```

这说明 umask 变了，创建时的默认权限也随之变化。

7.4 umask 的作用范围

7.4.1 当前进程属性

umask 是当前进程的属性，并会被子进程继承。

修改当前 Shell 的 umask：

```
umask 027
```

会影响该 Shell 之后创建的文件和目录，但不会追溯修改已经存在的文件。

7.4.2 想长期生效

如果希望每次登录都使用相同 umask，可以把设置写入用户的 Shell 启动文件，例如 `~/.bashrc`，或在系统级登录配置中设置。

修改前应确认目标 Shell 和登录方式，避免把配置写到不生效的文件中。

7.4.3 常见 umask

umask	普通文件常见结果	目录常见结果	特点
`022`	`644`	`755`	组用户可读，不能写

umask	普通文件常见结果	目录常见结果	特点
`027`	`640`	`750`	其他人基本不能访问
`002`	`664`	`775`	适合需要组内协作的场景
`077`	`600`	`700`	只有拥有者可访问

这些结果仍受起始权限和工具自身行为影响。例如有些程序会显式设置文件模式，而不是完全依赖 umask。

8. 共享目录与粘滞位

8.1 多人共享目录的问题

8.1.1 为什么不放在某人的家目录

家目录通常是私人空间。普通用户一般不应进入其他人的家目录，因此共享文件不适合放在：

```
/home/alice/shared
```

更合适的做法是使用系统级共享目录，例如：

```
/srv/share
/tmp
```

或管理员专门创建的共享路径。

8.1.2 只开放目录写权限会产生什么结果

如果共享目录允许所有用户写入，那么任何用户都可能在目录中：

- 创建自己的文件。
- 删除别人的文件。
- 重命名别人的文件。

从目录权限模型看，这是正常的；但从共享协作需求看，通常不希望别人删除自己的文件。

8.2 粘滞位解决什么问题

8.2.1 目标

目标是允许大家在共享目录中创建文件，但限制普通用户删除别人的文件。

创建共享目录并开放写权限：

```
sudo mkdir -p /srv/share
sudo chmod 1777 /srv/share
```

也可以分步设置：

```
sudo chmod o+w /srv/share
sudo chmod +t /srv/share
```

8.2.2 查看粘滞位

```
ls -ld /srv/share
```

可能看到：

```
drwxrwxrwt
```

最后一个位置的 `t` 表示 other 的执行位和执行权限上同时设置了粘滞位。

如果显示的是大写 `T`，通常表示设置了粘滞位，但没有给 other 执行权限。

8.2.3 谁的删除操作会被允许

在带粘滞位的共享目录中，通常只有以下角色可以删除或重命名某个文件：

1. root。
2. 共享目录的所有者。
3. 文件本身的所有者。

如果父目录没有相应的 `w+x` 权限，这些角色也不能完成删除。

8.2.4 粘滞位不是“把文件变成只读”

粘滞位主要限制目录中的删除、重命名等操作，不会自动改变文件内容的读写权限。

一个文件能否被读取、写入，仍要单独查看该文件本身的权限。

8.3 /tmp 为什么特殊

`/tmp` 是系统和应用经常使用的临时目录，通常允许许多用户写入，因此会启用粘滞位。

这使大家能够在其中创建临时文件，同时降低普通用户删除他人临时文件的风险。

需要长期共享的数据不应只放在 `/tmp`，因为系统清理策略可能删除临时内容。

9. 权限问题排查与课堂练习

9.1 权限被拒绝时怎么查

9.1.1 先确认身份

```
id
whoami
groups
```

需要知道当前进程使用的是哪个用户、哪些组。

9.1.2 再确认文件和目录

```
ls -l file
ls -ld dir
stat file
```

重点查看：

- 文件类型。

- owner、group、other 的权限。
- 文件拥有者和所属组。
- 当前用户命中哪一类角色。

9.1.3 检查完整路径

```
namei -l /path/to/file
```

路径上任意一级目录缺少 `x`，都可能让最终访问失败。

9.1.4 判断操作对象

如果操作是“读取文件内容”，检查文件本身的 `r`。

如果操作是“修改文件内容”，检查文件本身的 `w`。

如果操作是“执行程序”，检查文件本身的 `x` 和程序格式。

如果操作是“列出目录”，检查目录的 `r` 以及必要的 `x`。

如果操作是“删除或重命名文件”，检查父目录的 `w+x` 和粘滞位。

如果操作是“安装软件或修改系统配置”，检查目标系统目录和配置文件是否属于 root、当前普通用户是否确实需要 sudo。

9.2 三个典型场景

9.2.1 能列出目录，却不能进入

权限可能是：

```
dr--r--r--
```

当前用户能读取文件名单，却不能进入目录，因为没有 `x`。

9.2.2 文件可读，却无法执行

文件可能是：

```
-rw-r--r--
```

能读内容，但没有 `x`，所以不能直接执行。即使加上 `x`，还要确认文件格式是否合法。

9.2.3 文件本身不可写，却能被删除

文件可能是：

```
---------- 1 root root ... file
```

但父目录允许当前用户写入和执行，而且没有粘滞位。删除修改的是父目录中的目录项，不是文件内容。

9.3 权限判断的完整顺序

遇到复杂问题时，可以按下面的顺序分析：

```
当前用户是谁
    ↓
当前用户属于哪些组
    ↓
文件属于谁、哪个组
    ↓
当前用户命中 owner、group 还是 other
    ↓
检查该角色是否有对应 r/w/x
    ↓
检查路径上的父目录是否可穿过
    ↓
判断操作是文件内容操作还是目录项操作
    ↓
检查粘滞位等额外规则
    ↓
必要时判断是否需要 sudo
```

9.4 常用命令速查

9.4.1 查看身份与权限

```
id
groups
whoami
ls -l
ls -ld dir
stat file
namei -l /path/to/file
```

9.4.2 修改权限与归属

```
chmod u+x file
chmod g-w file
chmod o=r file
chmod 644 file
chmod 755 script.sh
chmod 700 private_dir
chmod -R 755 dir
sudo chown user file
sudo chown user:group file
chgrp group file
```

9.4.3 默认权限与共享目录

```
umask
umask 022
umask 000
sudo chmod 1777 /srv/share
sudo chmod +t /srv/share
ls -ld /srv/share
```

9.4.4 身份切换

```
su username
su - username
sudo command
sudo -l
```

```
sudo -i
exit
```

9.5 课堂自测问题

9.5.1 用户身份

1. 为什么 Linux 系统中通常只需要一个 root?
2. root 切换到普通用户时，为什么一般不需要普通用户密码?
3. `su -` 与 `su` 的环境有什么区别?
4. 为什么普通用户执行 sudo 通常输入自己的密码?
5. sudoers 的名单机制如何实现最小权限管理?

9.5.2 文件属性

1. 如何从 `ls -l` 中区分拥有者、所属组和 other?
2. 为什么文件有 `x` 权限仍然可能无法执行?
3. 如果拥有者和所属组是同一个人，为什么去掉 owner 权限后不能改用 group 权限?
4. root 为什么通常能够读取权限为 `000` 的文件?
5. `chmod`、`chown`、`chgrp` 分别修改什么?

9.5.3 目录与删除

1. 进入目录需要 `r` 还是 `x`?
2. 只有 `r` 没有 `x` 时，为什么不能真正访问目录中的文件?
3. 删除文件时，为什么文件本身没有写权限也可能被删除?
4. 为什么共享目录不应放在某个人的家目录里?
5. 粘滞位开启了以后，哪些人可以删除文件?

9.5.4 umask

1. 普通文件通常从 `666` 开始，目录通常从 `777` 开始，为什么?
2. 为什么 umask 最终计算使用按位运算而不是普通减法?
3. `umask 022` 时，新建文件和目录通常是什么权限?
4. `umask 000` 后，新建文件为什么可能出现 `rw-rw-rw-`?

9.6 本篇结论

Linux 权限不是一个单独的 `chmod` 命令，而是一条完整的判断链：

```
Shell 找到并执行命令
    ↓
当前用户以某个身份运行
    ↓
内核根据文件属性和用户角色判断权限
    ↓
通过则执行，不通过则 Permission denied
```

理解这条链路后，才能准确回答以下问题：

- 为什么命令可以执行，却不能修改命令文件？
- 为什么普通用户需要 sudo 安装软件？
- 为什么目录有 `r` 仍然不能进入？
- 为什么文件自己没有写权限，却可能被删除？
- 为什么修改 owner 或 group 会改变权限结果？
- 为什么新建文件的权限与 umask 有关？

本篇只整理权限部分。软件包管理器、vim、gcc/g++、make、进度条、Git 和 gdb/cgdb 将放入各自专题文件，避免不同主题互相打断。